

DineOS — Flutter Frontend Plan

Proposed architecture, screen scope, phasing, and tech stack — submitted for approval before any Flutter code is written.

STATUS: PROPOSAL — nothing below is built yet

1. What this covers

The backend (multi-tenant DineOS API — QR ordering, kitchen lifecycle, billing, Super Admin tenant control) is built and tested. This document proposes how the client side gets built on top of it: one Flutter codebase, three deployment targets, phased the same way the backend was — **core order flow first, management and platform tooling after**.

Four things need your sign-off, each marked **APPROVAL NEEDED** at its section below:

- 1. The 3-target split (Staff app / Customer web / Super Admin web) instead of one universal app
- 2. The tech stack
- 3. The phasing order
- 4. One small backend addition this plan depends on (Section 3)

2. Architecture — one codebase, three targets

Not every role needs the same delivery mechanism. A customer scans a QR code once and orders — they will not install an app for a single meal. A Super Admin manages tenants occasionally, from a desk, not a phone. Restaurant staff are the only group using the same device, daily, who benefit from a real installed app.



All three targets are **one Flutter codebase** (shared design system, models, API client) compiled three ways — not three separate projects. This avoids duplicating business logic while still giving each audience the right delivery format.

APPROVAL NEEDED — Architecture: confirm the 3-target split above. Alternative: a single universal app installed by everyone, including customers. Recommended against — it adds friction to the one flow (customer ordering) where friction directly costs the restaurant orders.

Tech stack

Concern	Choice	Why
Framework	Flutter 3.x / Dart	Already the agreed direction; one codebase, three targets above
State management	Riverpod	Testable without a widget tree, plain-Dart unit tests for business logic — matches the backend's test-first approach
Routing	go_router	Declarative, role-based redirect guards (e.g. Kitchen role never sees Admin routes)
HTTP client	dio	Interceptors for JWT attach/refresh and tenant headers in one place
Realtime	web_socket_channel	Thin wrapper over the existing Channels WS endpoints; auto-reconnect with backoff
Token storage	flutter_secure_storage	Mobile keychain/keystore; falls back to browser storage on the two web targets
Testing	flutter_test + integration_test	Widget tests per screen, integration tests against a running backend instance — mirrors the backend's pytest + Postman rigor

APPROVAL NEEDED — Tech stack: the table above. No blocking dependencies on a paid service (Riverpod, go_router, dio are all free/open-source).

3. One backend prerequisite this plan depends on

Gap found while writing this plan: the staff-facing `GET /v1/auth/me/` endpoint returns the logged-in user's own profile (email, name, role) but *not* their restaurant's feature flags (`kitchen_enabled` , `billing_enabled` , `notifications_enabled` , `realtime_enabled`). Those flags currently only exist on the Super-Admin-only `/platform/tenants/` endpoint. For the Staff app to hide a Kitchen tab or a Billing screen when a client hasn't paid for that add-on — the whole point of the modular pricing model — it needs a way to read its own tenant's flags at login. This is a small addition (extend `UserSerializer` / `MeView` to include the four flags via `request.user.restaurant`), not a redesign, but it's backend work that should land before Flutter screens start branching on flags.

4. Screen scope, by role

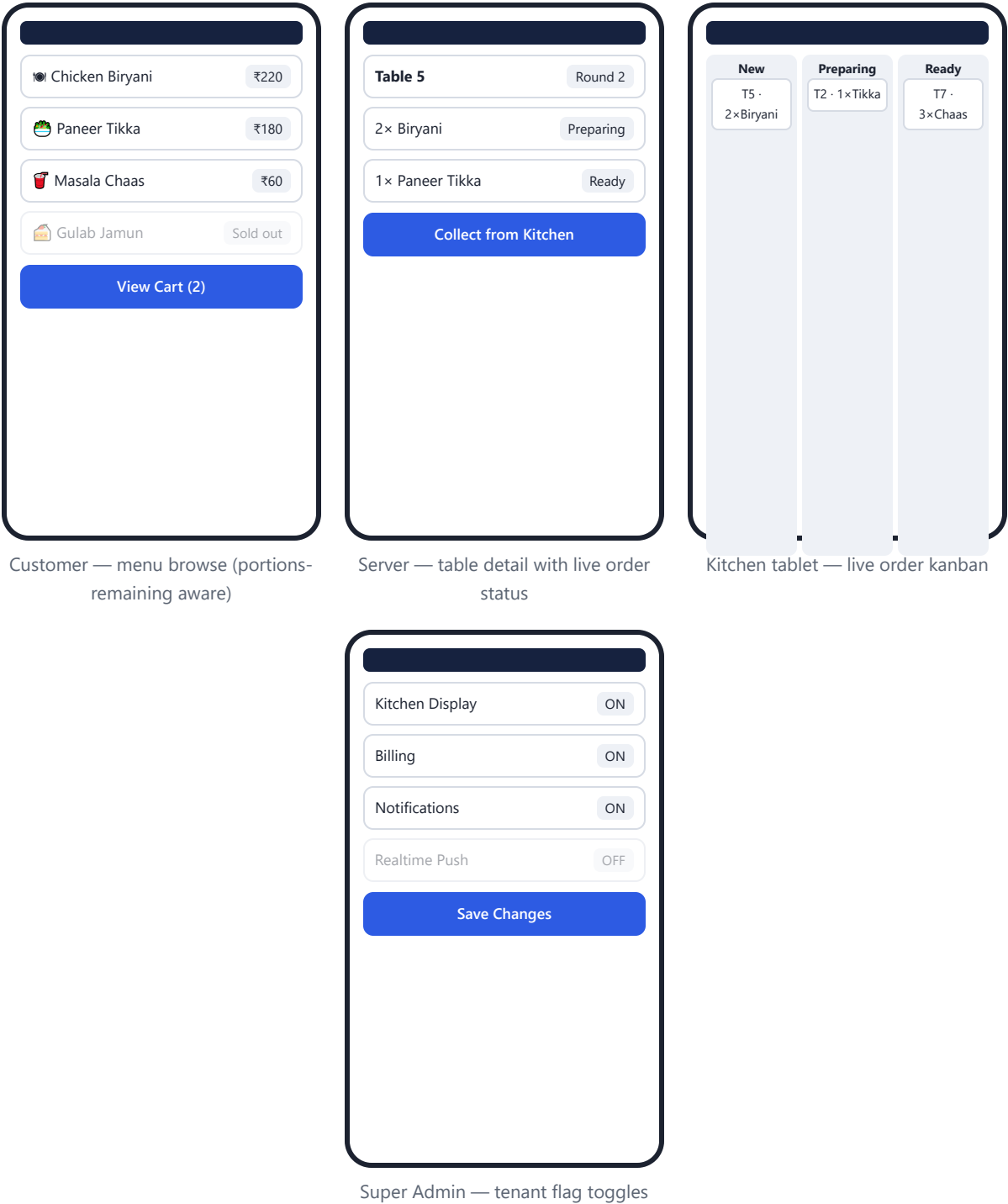
Counts below are grounded in the Phase-1 backend endpoints that actually exist today (verified against `dineos/urls.py` and each app's routes) — not a guess. Admin and Manager share one screen set, gated by permission rather than duplicated, since the backend already treats them that way (`IsAdminOrManager`).

Role	Delivery	Screens (approx.)	Core screens
Customer	Customer Web	9	Menu browse → cart → order status (live) → order history → request bill → payment status
Server	Staff App	8	Table floor view → table detail/session → take order → ready-to-collect queue → notifications
Cashier	Staff App	7	Bill-request queue → bill preview → confirm payment → notifications
Kitchen (KDS)	Staff App (tablet, device key — no personal login)	4	Device pairing (once) → live order kanban board → order detail
Admin / Manager	Staff App	14	Dashboard → staff management → menu & category management → prepared-portions → table history/reports → notifications
Super Admin	Super Admin Web	5	Tenant list → tenant detail → create tenant → flag toggles

~47 screens total. The figure discussed earlier (~53) included Phase 2 backend features (inventory, AI insights, customer feedback, cash-count/shift reports) that don't exist in the API yet — their screens aren't scoped here and will be added once that backend work lands, so the two numbers aren't in conflict.

Preview — indicative wireframes

Layout only, to make the plan concrete — not final visual design. Actual look/branding will be pulled screen-by-screen from the Figma file once a phase starts.



5. Feature-flag-aware UI

The Staff app reads its tenant's four flags once at login (Section 3) and adapts, mirroring exactly what the backend already enforces server-side — the UI never shows an action the API would reject:

Flag off	Staff app behavior
kitchen_enabled	Kitchen tab/board hidden entirely; Server's "take order" flow skips the "sent to kitchen" status and shows orders as served immediately
notifications_enabled	Notification bell/badge hidden across all staff screens
billing_enabled	Cashier's bill screens hidden; Server's "request bill" action hidden on the table view
realtime_enabled	App falls back to manual pull-to-refresh instead of opening a WebSocket; no live badge/indicator shown

6. Phasing plan

Same philosophy as the backend build: **the core paid-for loop first**, management tooling once that loop is proven end-to-end on a device.

Phase	Scope	Depends on
0	Backend flags-on-login addition (Section 3); shared API client, auth, design tokens, routing skeleton for all 3 targets	—
1	Customer Web + Server screens — QR scan through order placement through live status. The core revenue loop, provable on a real table.	Phase 0
2	Kitchen tablet + Cashier — completes the order lifecycle (kitchen prep) and closes the loop (payment)	Phase 1
3	Admin/Manager — staff management, menu & category management, prepared- portions, reports	Phase 1
4	Super Admin Web — tenant list/create/flag toggles	Phase 0 (independent of 1–3)

APPROVAL NEEDED — Phasing order: the table above. Phase 4 (Super Admin Web) has no dependency on Phases 1–3 and could run in parallel if there's bandwidth to build it alongside Phase 1.

7. Testing strategy

- **Widget tests** per screen — same rigor as the backend's pytest suite, run in CI before merge.
- **Integration tests** (`integration_test` package) driving the app against a real running backend instance (the same throwaway-tenant pattern used for the Postman runs) — covers the full QR-scan-to-payment loop, not just isolated widgets.
- **Manual device pass** at the end of each phase: one physical Android device + one iOS device, the actual golden-path flow for that phase's roles.

8. Explicitly out of scope for v1

- Offline mode / poor-connectivity queuing — v1 assumes the venue has working Wi-Fi, same assumption the backend already makes
- Push notifications via FCM/APNs — v1 uses in-app WebSocket + notification-list polling only, matching what the backend currently emits
- Any Phase-2 backend feature (inventory, AI insights, feedback, cash-count) — no frontend screens until that backend work exists

DineOS — Flutter Frontend Plan · proposal, nothing built yet · pending approval